

```

# BLOB Principle - Protocole D - Caracterisation thermalisation profonde
# Version protegee - ASCII pur - Auto-test integre
#
# Objectif : determiner le vrai plateau de n_tri(Blob) a N=50000 fixe
# en allongeant progressivement le burn-in jusqu a observer la stabilisation
#
# Protocole C precedent a montre :
#   burn=5N -> n_tri=526
#   burn=10N -> n_tri=828
#   burn=20N -> n_tri=1436
# La croissance n est pas saturee. Il faut aller plus loin.

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous_chars = {
                chr(8216): "U+2018 (apostrophe gauche)",
                chr(8217): "U+2019 (apostrophe droite)",
                chr(8220): "U+201C (guillemet gauche)",
                chr(8221): "U+201D (guillemet droit)",
                chr(8211): "U+2013 (en-dash)",
                chr(8212): "U+2014 (em-dash)",
            }
            found = []
            for c, name in dangerous_chars.items():
                if c in content:
                    found.append((name, content.count(c)))
            if found:
                print("ERREUR : caracteres corrompus")

```

```

        for name, count in found:
            print(" - " + name + " : " + str(count))
        return False
    star = chr(42)
    bad_init = star + star + "init" + star + star
    if content.count(bad_init) > 1:
        print("ERREUR : motif corrompu detecte au lieu de double-undersco
        return False
    triple_back = chr(96) + chr(96) + chr(96)
    if content.count(triple_back) > 1:
        print("ERREUR : delimitateurs Markdown detectes")
        return False
    print("OK : aucun caractere corrompu")
    print("OK : __init__ correct")
    print("OK : pas de Markdown")
else:
    print("Note : pas de verification source possible (ok si exec via ope
except Exception as e:
    print("Note : auto-test partiel : " + str(e))
print("=" * 70)
return True

if not auto_check():
    print("ARRET : corriger les caracteres avant de continuer.")
    sys.exit(1)

# =====
# IMPORTS
# =====
import numpy as np
import time
from collections import defaultdict

# =====
# Parametres canoniques
# =====
KAPPA_0 = 8
BETA = 0.30
TAU = 0.50

# =====
# Classe BlobGraph (identique aux scripts precedents)
# =====
class BlobGraph:

```

```

def __init__(self, N, seed, is_null=False):
    self.N = N
    self.is_null = is_null
    self.rng = np.random.default_rng(seed)
    self.E_target = int(N * KAPPA_0 / 2)
    self.adj = [set() for _ in range(N)]
    self.edge_list = []
    self._init_graph()
    self.deg = np.array([len(self.adj[i]) for i in range(N)], dtype=float)
    self.n_tri = self._count_triangles()
    self.S_curr = self._compute_S(self.n_tri)

def _init_graph(self):
    for i in range(self.N):
        u = i
        v = (i + 1) % self.N
        if v not in self.adj[u]:
            self.adj[u].add(v)
            self.adj[v].add(u)
            self.edge_list.append((min(u, v), max(u, v)))
    while len(self.edge_list) < self.E_target:
        u = int(self.rng.integers(self.N))
        v = int(self.rng.integers(self.N))
        if u != v and v not in self.adj[u]:
            self.adj[u].add(v)
            self.adj[v].add(u)
            self.edge_list.append((min(u, v), max(u, v)))

def _count_triangles(self):
    tri = sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list)
    return tri // 3

def _compute_S(self, n_tri_val):
    mu = self.deg.mean()
    D = self.deg.std() / mu if mu > 0 else 0.0
    return np.log(1 + n_tri_val) - BETA * D

def step_blob(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False

```

```

dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = self._compute_S(new_n_tri)
log_alpha = self.N * (new_S - self.S_curr) / TAU
if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.S_curr = new_S
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False

def step_null(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))

```

```

v_add = int(self.rng.integers(self.N))
if u_add == v_add or v_add in self.adj[u_add]:
    return False
u_add, v_add = min(u_add, v_add), max(u_add, v_add)
if (u_add, v_add) == (u_rm, v_rm):
    return False
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
self.edge_list[idx] = (u_add, v_add)
self.n_tri = new_n_tri
self.S_curr = self._compute_S(new_n_tri)
return True

```

```

# =====
# SIMULATION AVEC TRACE TEMPORELLE FINE
# =====

```

```

def run_with_trace(N, seed, total_steps, is_null, label_prefix):

```

```

    """

```

```

    Au lieu de separer burn et production, on enregistre n_tri tout au long
    de la simulation. Cela permet de detecter visuellement la stabilisation.

```

```

    """

```

```

    t_start = time.time()

```

```

    type_label = 'NULL' if is_null else 'BLOB'

```

```

    log_id = ' [' + label_prefix + type_label + ' N=' + str(N) + ' s=' + str(seed)
    print(log_id + ' Start ' + str(total_steps) + ' steps...', flush=True)

```

```

    sim = BlobGraph(N=N, seed=seed, is_null=is_null)

```

```

step_func = sim.step_null if is_null else sim.step_blob

# Echantillonnage : 100 points repartis logarithmiquement
n_samples = 100
sample_steps = sorted(set([
    int(np.exp(np.log(total_steps) * i / n_samples))
    for i in range(1, n_samples + 1)
]))

trace = [] # liste de (step, n_tri, acc_cumul, S)
acc = 0
next_idx = 0

for i in range(total_steps):
    if step_func():
        acc += 1
    if next_idx < len(sample_steps) and (i + 1) >= sample_steps[next_idx]:
        trace.append({
            'step': i + 1,
            'n_tri': sim.n_tri,
            'acc': acc / (i + 1),
            'S': sim.S_curr
        })
        next_idx += 1
    # Print intermediaire tous les 20 echantillons
    if next_idx % 20 == 0:
        print('    step ' + str(i+1) + '/' + str(total_steps)
              + ' n_tri=' + str(sim.n_tri)
              + ' acc=' + '{:.3f}'.format(acc / (i+1))
              + ' t=' + '{:.0f}'.format(time.time() - t_start) + 's', flu

total_time = time.time() - t_start
final_acc = acc / total_steps

# Analyse de stabilisation : derniere quartile
last_q = trace[3 * len(trace) // 4:]
ntri_last_q = [t['n_tri'] for t in last_q]
ntri_last_mean = np.mean(ntri_last_q)
ntri_last_std = np.std(ntri_last_q)
rel_var_last_q = ntri_last_std / ntri_last_mean if ntri_last_mean > 0 else 0

# Derive sur derniere moitie
half = len(trace) // 2
n_first = np.mean([t['n_tri'] for t in trace[half:3*half//2]])
n_second = np.mean([t['n_tri'] for t in trace[3*half//2:]])
drift_rel = abs(n_second - n_first) / n_first if n_first > 0 else 0

```

```

print(log_id + ' Done. n_tri_final=' + str(sim.n_tri)
      + ' n_tri_last_q=' + '{:.1f}'.format(ntri_last_mean)
      + ' rel_var=' + '{:.2%}'.format(rel_var_last_q)
      + ' drift=' + '{:.2%}'.format(drift_rel)
      + ' acc=' + '{:.3f}'.format(final_acc)
      + ' t=' + '{:.0f}'.format(total_time) + 's', flush=True)

return {
    'type': type_label,
    'N': N,
    'seed': seed,
    'total_steps': total_steps,
    'n_tri_final': int(sim.n_tri),
    'n_tri_last_q_mean': float(ntri_last_mean),
    'n_tri_last_q_std': float(ntri_last_std),
    'rel_var_last_q': float(rel_var_last_q),
    'drift_rel_halves': float(drift_rel),
    'acc_global': float(final_acc),
    'time_sec': float(total_time),
    'trace': trace
}

# =====
# PROTOCOLE D : Thermalisation profonde a N=50000
# =====
print('')
print('=' * 75)
print('  PROTOCOLE D : Thermalisation profonde N=50000')
print('=' * 75)
print('  Protocole C precedent : n_tri continue de croitre avec burn')
print('  burn=5N -> 526, burn=10N -> 828, burn=20N -> 1436')
print('')
print('  Ici, nous testons des durees encore plus longues')
print('  jusqu a observer la VRAIE saturation (rel_var sur derniere quartile < 5%)')
print('=' * 75)

N_FIXED = 50000

# Strategie : 3 niveaux de profondeur, avec acc dans la derniere quartile = crite
# Niveau 1 : 30N (1.5M pas) - environ 4-5 min
# Niveau 2 : 50N (2.5M pas) - environ 7-8 min
# Niveau 3 : 100N (5M pas) - environ 15 min
# 2 seeds par niveau Blob, 1 seed Null par niveau pour reference

LEVELS = [
    {'name': '30N', 'steps': 30 * N_FIXED, 'seeds': 2},

```

```

        {'name': '50N', 'steps': 50 * N_FIXED, 'seeds': 2},
        {'name': '100N', 'steps': 100 * N_FIXED, 'seeds': 2},
    ]

print('')
print('  Niveaux : ' + ', '.join([l['name'] for l in LEVELS]))
print('  Seeds Blob par niveau : 2')
print('  Seeds Null par niveau : 1 (reference)')
print('')
total_sims = sum(l['seeds'] + 1 for l in LEVELS)
print('  Total simulations : ' + str(total_sims))
print('  Duree estimee : 35-50 minutes')
print('=' * 75)

t0 = time.time()
all_results = []

for level in LEVELS:
    print('')
    print('--- Niveau ' + level['name'] + ' (' + str(level['steps']) + ' pas) ---')

    # Seeds Blob
    for seed in range(level['seeds']):
        r = run_with_trace(N_FIXED, seed, level['steps'], is_null=False, label_pr
        r['level'] = level['name']
        all_results.append(r)

    # Seed Null (reference)
    r = run_with_trace(N_FIXED, 100, level['steps'], is_null=True, label_prefix='
    r['level'] = level['name']
    all_results.append(r)

total_time = time.time() - t0

# =====
# RAPPORT FINAL DETAILLE
# =====
print('')
print('=' * 75)
print('  RAPPORT PROTOCOLE D')
print('=' * 75)
print('  Temps total : ' + '{:.1f}'.format(total_time / 60) + ' min')
print('')

# Tableau resume par niveau
print('  Resume par niveau (Blob seulement) :')
```

```

header = ' ' + 'level'.ljust(8) + 'steps'.ljust(12) + 'n_tri_q4'.ljust(12)
header += 'rel_var_q4'.ljust(13) + 'drift_halves'.ljust(15) + 'acc'
print(header)
print(' ' + '-' * 70)

by_level = defaultdict(list)
for r in all_results:
    if r['type'] == 'BLOB':
        by_level[r['level']].append(r)

ntri_by_level = []
for level in LEVELS:
    rs = by_level[level['name']]
    n_q4 = np.mean([r['n_tri_last_q_mean'] for r in rs])
    rv_q4 = np.mean([r['rel_var_last_q'] for r in rs])
    drift = np.mean([r['drift_rel_halves'] for r in rs])
    acc = np.mean([r['acc_global'] for r in rs])
    ntri_by_level.append((level['name'], n_q4))
    line = ' ' + level['name'].ljust(8)
    line += str(level['steps']).ljust(12)
    line += '{:.1f}'.format(n_q4).ljust(12)
    line += '{:.2%}'.format(rv_q4).ljust(13)
    line += '{:.2%}'.format(drift).ljust(15)
    line += '{:.3f}'.format(acc)
    print(line)

# Reference Null
print('')
print(' Reference Null (1 seed par niveau) :')
print(' ' + 'level'.ljust(8) + 'n_tri_q4'.ljust(12) + 'rel_var_q4')
for r in all_results:
    if r['type'] == 'NULL':
        print(' ' + r['level'].ljust(8) + '{:.1f}'.format(r['n_tri_last_q_mean'])
              + '{:.2%}'.format(r['rel_var_last_q']))

# Verdict
print('')
print('=' * 75)
print(' VERDICT')
print('=' * 75)

# Croissance entre niveaux
ntri_30 = ntri_by_level[0][1]
ntri_50 = ntri_by_level[1][1]
ntri_100 = ntri_by_level[2][1]
growth_30_50 = (ntri_50 - ntri_30) / ntri_30
growth_50_100 = (ntri_100 - ntri_50) / ntri_50

```

```

print('  Croissance n_tri 30N -> 50N : ' + '{:+.1%}'.format(growth_30_50))
print('  Croissance n_tri 50N -> 100N : ' + '{:+.1%}'.format(growth_50_100))
print('')

if abs(growth_50_100) < 0.05:
    print('  -> SATURATION ATTEINTE a 50N-100N')
    print('      Le vrai plateau de n_tri est environ ' + '{:.0f}'.format(ntri_100))
    print('      [M FORT] : valeur de reference a inscrire au manifeste')
elif abs(growth_50_100) < 0.15:
    print('  -> Saturation approchee mais pas complete')
    print('      Necessite extension a 200N pour confirmation finale')
elif growth_50_100 > 0.30:
    print('  -> Croissance toujours forte : pas de saturation observee')
    print('      n_tri pourrait croitre lineairement avec le temps de simulation')
    print('      Ou avoir besoin de 500N+ pour saturer')
    print('      [H] : revoir le modele - peut-etre pas de plateau du tout')
else:
    print('  -> Regime intermediaire, ralentissement progressif')

# Comparaison Blob vs Null au niveau 100N
print('')
null_100 = [r for r in all_results if r['type'] == 'NULL' and r['level'] == '100N']
if null_100:
    null_n = null_100[0]['n_tri_last_q_mean']
    blob_n = ntri_100
    ratio = blob_n / null_n if null_n > 0 else 0
    print('  Ratio Blob/Null a thermalisation profonde (100N) : ' + '{:.2f}x'.format(ratio))
    print('  Comparaison avec ratios precedents :')
    print('      N=25k burn=10N -> ratio 16.6x (sous-thermalise probable)')
    print('      N=50k burn=5N -> ratio 6.3x (sous-thermalise)')
    print('      N=50k burn=100N -> ratio ' + '{:.2f}x'.format(ratio) + ' (vraiment')

print('')
print('=' * 75)
print('  FIN PROTOCOLE D')
print('=' * 75)

```